uc3m | Universidad Carlos III de Madrid

University Degree in Telematics Engineering
2024-2025

Bachelor Thesis

"Machine Learning-Based Predictive Modeling of Energy Prices"

Rodrigo De Lama Fernández

Emilio Parrado Hernández

Madrid, España, May 31st 2025

SUMMARY

This project focuses on the development of a machine learning-based predictive model for electricity prices in Spain. Using historical data from the OMIE and technical analysis (TA) indicators, the model aims to accurately forecast hourly energy prices. The study focuses on a single hourly slot, evaluating the performance of various Machine Learning models, such as linear regression, Lasso, and Random Forest. There was a strong focus on the feature engineering, employing technical analysis indicators such as moving averages, exponential moving averages, and momentum metrics. The results display the impact of tailoring the features to improve model accuracy and offer insights into the potential of data-driven approaches for energy price forecasting.

Keywords:

Energy

Machine Learning

Sliding Window

Technical Analysis (TA)

DEDICATION

Dedicated to my late grandfather, who was not able to see me become an engineer like himself.
To my amazing family.
To my parents, that have always helped me push through hard moments.
To my grandparents who will proudly see me become an engineer.
To my amazing girlfriend that has supported me though all the ups and downs of life.
To my all of friends and colleagues that have accompanied me these years.
To anyone and everyone that has supported me during my years in university.

Here is to the next steps in life

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION TO THE PROBLEM

- **1. Explanation of how the price of energy works in Spain**
  - a. Energy
  - b. Prices
- **2. Machine Learning - Lets run through some model overviews, which are available to choose, but without actually referencing anything**

Energy has increasingly become more important. Availability is key for a modern high functioning society In Europe we are lucky to have a really advanced and reliable network. It is a key resource needed to succeed in the ecological transition that we are currently undergoing world wide. Electric mobility is a key element of this process. Electric cars, buses and trucks, need easily available and convenient sources of energy to recharge, to be able to play their role in society. An important factor in this ever growing necessity is the price of the available energy. But in recent times, that easy and affordable access has changed, and has become more unpredictable. Ever since the commencement of the war between Russia and Ukraine marked and inflection point in the mostly cyclical nature of electricity prices. Prices have gone up considerable since a couple years time, and it has been affecting everyone, from big consumers such as companies, to everyday people, having to pay a more expensive electricity bill at the end of the month. This has become a topic of great concern, with more and more people thinking about it often.

This led me to think about pricing, how it was structured, and what to expect as an end consumer. Could we plan our use of energy according to the price? Could it be predicted accurately? These were some of the questions I had that sparked my interest in looking into energy predictions.

In the initial discussions of this project, we talked about various ways to attack the problem at hand. We thought about a variety of systems but settled on a plan to explore the predictions with the more established methods of Machine Learning. This was done specially since it was a natural continuation of what was learned in the third year subject of Telematics Engineering, Modern Theory of Detection and Estimation.

The first idea we developed was to separate out the project in 24 blocks, one for each hour block to predict. The reason behind splitting the data in 24 blocks was simple, we assumed that data between the same time slot across neighbor days, would be similar, or would follow a certain trend.

Focusing on the innovation side of things, we decided to simplify the problem. Pivoting from creating an algorithm to predict hourly prices, to splitting that into the 24 blocks, and predicting a single one. We decided on the 14^{th} hour of each day, since irrespective of the day of the week, or holiday, it would always be a valley period.

This kind of granular breaking up of data is similar to what a Random Forest does in order to absorb more details and create higher resolution predictions. We made this assumption based on the idea that it would simplify the project substantially.

Talk about Emilio's background in investment banking at BBVA and his alorithms.

Talk about the idea of using technical analysis for feature engineering

2. BACKGROUND AND RELATED WORK

2.1. General background

How is the price of electricity set in Spain and Portugal?

LEER ARTICULO [1]

What happens when new producers enter the market and sell to the energy pool?

Comment about more typical case studies focusing on energy consumption, since that is something more consistent and predictable

2.2. The Sliding Window plan: Setting up the Feature Matrix

How do I prepare my data to utilize across different models: The Feature Matrix (weight matrix)

The sliding window theory (mini-models)

2.3. Which Machine Learning models will be used

What algorithms will I use?

- Linear Regression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

- Lasso Regression: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html

- Random Forest: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

2.4. Fine Tuning with Technical Analysis and Feature engineering

What is *Fine-tuning?* - https://www.ibm.com/es-es/think/topics/fine-tuning

What Type of Fine-tuning will I use?

- Feature Engineering & Feature Selection

- Technical analysis based feature engineering

What is Technical Analysis? - https://www.britannica.com/money/what-is-technical-analy

Technical indicators

Que es cada cosa y porque usarlos en vez de series temporales, porque la info temporal esta implicita en los indicadores.

Indicadores que capturan patrones en la serie

Ventaja: modelo ML mas sencillo pq las features son muy informativas vs

TA metrics used? SMA, EMA, ROC & RSI

Talk about the intervals used:

$SMA_3, SMA_5, SMA_7, SMA_{14}, SMA_{30}, SMA_{60}, SMA_{90}, SMA_{180}, SMA_{360}$

$EMA_3, EMA_5, EMA_7, EMA_{14}, EMA_{30}$

$ROC_3, ROC_5, ROC_7, ROC_{12}, ROC_{14}, ROC_{30}$

RSI_5, RSI_7, RSI_{14}

2.5. Evaluation Metrics

How will I measure my accuracy?

errors:

R^2

RMSE & MSE

2.6. Related Work

Other ways of predicting

Temporal Series

Energy consumption predictions

3. PROPOSAL: THEORETICAL SYSTEM DESCRIPTION

This chapter will outline the design of the system developed for the project, describing the complete pipeline constructed for the system. This will be explained starting from a blank state, detailing the complete set-up process required to run the project. Following this initial configuration, the data preparation stage will be outlined. This will be continued by the feature matrix creation process, concluding with the training of the machine learning models and the chosen evaluation methods.

The following Block diagram will model in the most basic forms the system design:

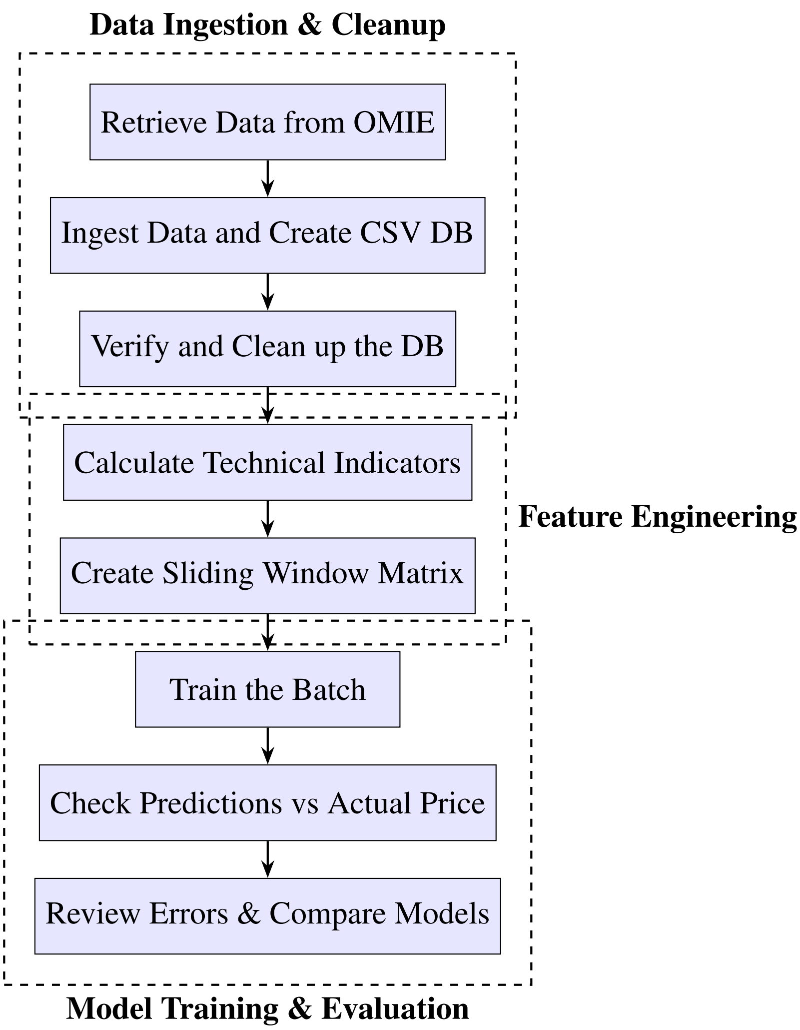

Fig. 3.1. Theoretical Block Diagram of the Project Pipeline

3.1. Prior Set Up and Configuration

Before getting into the system explanation, there are some important pre-requisites that must be met in order to run the project. The project was developed and tested with Python 3.13 [2], so for any recreation of the project, it is highly recommendable to use the same version. For additional assurance of reproducibility, and as a universal best practice for Python development, it is strongly advisable to set up a new virtual environment [3]. Alternatively, another system that would achieve the same goal of reproducibility, and dependency conflict avoidance would be the use of Miniconda [4]. In this project we chose to use the built in Python module *venv*, with which a virtual environment can be easily created and configured with the project dependencies using the following commands.

Create the virtual environment:

```
python -m venv .venv
```

Activating the new virtual environment:

```
in macOS
source .venv/bin/activate

in Windows
.venv\Scripts\Activate.ps1
```

To reproduce the exact environment used in the project, a *requirements.txt* file must be created with the following contents:

```
ipykernel==6.29.5
matplotlib==3.10.1
numpy==2.2.3
pandas==2.2.3
requests==2.32.3
scikit-learn==1.6.1
ta==0.11.0
```

To install the dependencies run:

```
pip install -r requirements.txt
```

These libraries were specifically selected to aid in the development of the project. Most of the project was written in Jupyter Notebooks, which are dependent on the *ipykernel* package. To aid in the retrieval of the data, the *requests* package was used to execute HTTP requests. For the manipulation of the data points, *pandas* was used to house the data structures, *numpy* for mathematical operations using arrays, and the *ta* technical analysis library was used to compute the technical indicators. Finally the machine learning algorithms were provided by the *scikit-learn* library. A special thank you is in order, dedicated to all the open source contributors of these libraries that have made this project possible

Following the completion of this prior set up required for the project, the following sections will detail in depth the different steps involved in the system pipeline.

3.2. Data Preparation: Ingestion and Cleanup

The first step in the data preparation process would be to retrieve the raw data from the original source, in this case this means downloading it from the OMIE. It is publicly available data easily discoverable through their webpage [5]. The data collected was all that was initially available towards the end of 2024, with some extraordinary retrievals done later on in 2025 to complete the set with more data points.

In this scenario, it meant retrieving data from 2018 onward. While for the 5 first years available the data was self contained in a compressed archive file, the rest from 2023 to our current 2025 was not. This meant that to retrieve the files, manually downloading each resource was required. In order to streamline this process, a short Python script was created to build the necessary URIs to retrieve the files with an HTTP GET request.

The following Python script was written with modularity in mid for ease of use. It firstly creates the filenames according to the OMIE's file naming convention, inserting them into a list. This list is essential to successfully build the URIs with which the HTTP GET request will be made. Finally the function `download_files(urls)` makes the request, retrieving the files.

```
import requests
from datetime import datetime, timedelta

# Usage instructions:
    # Set file name to save the filenames to download
    # Set start and end dates

file_path = 'to_download.txt'
start_date = datetime(2023, 1, 1)
end_date = datetime(2025, 3, 18)

# Compose filenames to download
def compose_filenames():
    start_date = start_date
    end_date = end_date

    # Generate the entries for the number of days comprehended
    to_generate = (end_date - start_date).days + 1
```

```
    entries = []
    for i in range(0, to_generate):  # x days from start to end
        date_entry = start_date + timedelta(days=i)
        entries.append(f'marginalpdbc_{date_entry.strftime("%Y%m%d")}.1')

    # Save to a .txt file
    with open(file_path, 'w') as file:
        for entry in entries:
            file.write(entry + '\n')

# Example URI to build
# https://www.omie.es/es/file-download?parents%5B0%5D=marginalpdbc&filename=marginalpdbc_20230102.1
def read_filenames_and_compose_urls(file_path):
    base_url = "https://www.omie.es/es/file-download?parents%5B0%5D=marginalpdbc&filename="

    # Read the filenames from the .txt file
    with open(file_path, 'r') as file:
        filenames = [line.strip() for line in file if line.strip()]

    # Compose the URLs
    urls = [base_url + filename for filename in filenames]

    return urls

def download_files(urls):
    for url in urls:
        # Extract the filename from the URL
        filename = url.split('=')[-1]

        # Send a GET request to download the file
        response = requests.get(url)

        # Check if the request was successful and write to file
        if response.status_code == 200:
            # Write the content to a file with the extracted filename
            with open(filename, 'wb') as file:
                file.write(response.content)
            print(f"Downloaded {filename}")
        else:
            print(f"Failed to download {filename}")

# Execution
compose_filenames()
urls = read_filenames_and_compose_urls(file_path)
download_files(urls)
```

The newly downloaded raw data was manually organized into its corresponding folder per year. An example filename is the following: `marginalpdbc_20230102.1` This would be an important organization step for the ingestion of the data, as the next procedure involves a script that iterates through these folders, identifying all files that start with `marginalpdbc_`, and parsing them into an easier to use form, a Pandas DataFrame [6].

When a valid file is identified while iterating through the year folders, the script reads its content. It will perform some cleanup steps, such as removing the first and last rows which are a standard header and footer across all files. It then parses the remaining data, which is separated by semicolons, into a Pandas DataFrame for that specific day. It will name the columns to *Year*, *Month*, *Day*, *Hour*, *MarginalPT* (Portuguese Energy Price), and *MarginalES* (Spanish Energy Price). It will also filter out any rows where the 'Year' column is not a digit, ensuring data integrity. The columns are then converted to their appropriate data types (integers for date/time components, floats for marginal prices). Each processed daily dataset is then temporarily stored in a list.

After processing all the individual files, the script concatenates all the daily data from the list of DataFrames into a single, comprehensive DataFrame. It will then construct a *Datetime* column built from the *Year*, *Month*, *Day*, and *Hour* columns. It will also handle some peculiar cases where an *Hour* value of 25 might appear (likely due to daylight savings time adjustments) by converting it to 0. This *Datetime* column is then set as the DataFrame's index. Finally, it removes the individual date and hour columns as well as the *MarginalPT* column, leaving only the *MarginalES* as the primary target variable, along with the *Datetime* index.

This clean and combined dataset will be saved as a CSV file for ease of use, `raw_data.csv`. The script then performs further data cleanup by reloading the `raw_data.csv`, and sorting it by *Datetime*, saving it as `processed_data.csv`. The whole process concludes by verifying the completeness of the hourly timestamps within the processed data, generating the full range of hourly timestamps that should be there, and checking for any discrepancies. If no missing timestamps are found, it confirms successful data processing; otherwise, it reports the missing timestamps, indicating an error in the data collection or processing.

For the previous in-depth explanation, you may find the following block of Python code that was created for the desired ingestion and cleanup tasks:

```python
import pandas as pd
import os

# Base path to the folder containing the year by year data folders
base_path = '../TFG/data/'

# Initialize an empty list to store DataFrames
all_data = []

# Iterate through each year folder and process all files
for root, dirs, files in os.walk(base_path):
    for file in files:
        # Check if the file starts with 'marginalpdbc_' (ignorering the rest, including the extension)
        if file.startswith('marginalpdbc_'):
```

```python

            # Read the file
            file_path = os.path.join(root, file)
            with open(file_path, 'r', encoding='utf-8') as f:
                lines = f.readlines()

            # Remove the first line (MARGINALPDBC;) and the last line (*)
            data_lines = lines[1:-1]

            # Convert the list of semicolon-separated strings into a DataFrame
            daily_data = pd.DataFrame([line.strip().split(';') for line in data_lines])

            # Remove trailing ';' from the last column
            if daily_data.shape[1] > 6:
                # Drop empty columns
                daily_data = daily_data.iloc[:, :6]

            # Assign column names
            daily_data.columns = ['Year', 'Month', 'Day', 'Hour', 'MarginalPT', 'MarginalES']

            # Remove rows with invalid data (in case accidental empty rows or non-numeric values get included)
            daily_data = daily_data[daily_data['Year'].str.isdigit()]  # Only keep rows where 'Year' is a number

            # Convert the columns to the appropriate data types
            daily_data = daily_data.astype({
                'Year': int, 'Month': int, 'Day': int, 'Hour': int, 'MarginalPT': float, 'MarginalES': float
                })

            # Append daily data to the list
            all_data.append(daily_data)

# Concatenate all daily DataFrames into one big DataFrame
full_data = pd.concat(all_data, ignore_index=True)

# Create a datetime column from Year, Month, Day, and Hour
# Adjust the 'Hour' column to deal with the potential 25th hour issue
full_data['Hour'] = full_data['Hour'].apply(lambda x: 0 if x == 25 else x)
full_data['Datetime'] = pd.to_datetime(full_data[['Year', 'Month', 'Day', 'Hour']], errors='coerce')

# Set the 'Datetime' column as the index
full_data.set_index('Datetime', inplace=True)
```

```

# Drop unnecessary columns
# Our target variable is 'MarginalES', so 'MarginalPT' will be dropped
full_data.drop(['Year', 'Month', 'Day', 'Hour', 'MarginalPT'], axis=1, inplace=True)

# Save the full dataset to a CSV file
full_data.to_csv('../../data/raw_data.csv')

# Database cleanup
# Load the dataset
df = pd.read_csv('../../data/raw_data.csv')

# Sort the data by the 'Datetime' column
df = df.sort_values(by='Datetime')

# Save the sorted data to a new file
df.to_csv('../../data/processed_data.csv', index=False)

# Read the CSV file
df = pd.read_csv('../../data/processed_data.csv')

# Convert 'Datetime' column to datetime objects
df['Datetime'] = pd.to_datetime(df['Datetime'])

# Generate the complete range of hourly timestamps
full_range = pd.date_range(start=df['Datetime'].min(), end=df['Datetime'].max(), freq='h')

# Find any missing timestamp
missing_timestamps = full_range.difference(df['Datetime'])

# If successful, print a message
if missing_timestamps.empty:
    print("Data processing completed successfully.")

# Else, print an error message
else:
    print("Error: Missing timestamps found. Check the data.")

    # Print missing timestamps
    print("Missing timestamps:")
    print(missing_timestamps)

    print("Data processing completed with errors.")
```

After completing this ingestion and cleanup process, the dataset will be reduced to just the 14^{th} hour data point. The following code will create a the final database file, which will house the singular *Datetime* value, and the *MarginalES* data point:

```python
import pandas as pd

# Load the original dataset
data_path = '../../data/processed_data.csv'
df = pd.read_csv(data_path, parse_dates=['Datetime'])

# Define the hour to predict and filter the data down to the 14:00H
    data point
hour_to_predict = 14
df_hour = df[df['Datetime'].dt.hour == hour_to_predict].copy()

# Save the final database
df_hour.to_csv("../../data/hour_14_metrics.csv", index=False)
```

3.3. The Feature Matrix Creation

Theory behind feature matrices:

The feature matrix creation consists on building one unique matrix that can be used across any desired model.

To define the **feature matrix**, often denoted as **X**, and the **target vector**, **y**:

For a given feature matrix containing only prices:

$$\mathbf{X} = \begin{Bmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \end{Bmatrix}$$

Its target target vector shall be:

$$\mathbf{y} = \begin{Bmatrix} y_1 \\ y_2 \\ y_3 \end{Bmatrix}$$

The relationships between the features and targets is the following:

$$x_4 = y_1$$

$$x_5 = y_2$$

$$x_6 = y_3$$

These relationships indicate a dependency where future features (x_4, x_5, x_6) are directly determined by past target values (y_1, y_2, y_3). This is common in time-series data or sequence prediction tasks, where the output of one step becomes an input for a subsequent step.

For the more complex feature matrix that includes the technical analysis indicators, which will be noted as *ta*, it will take the following shape:

$$\mathbf{X} = \begin{Bmatrix} x_1 & x_2 & x_3 & ta_{11} & ta_{21} & ... & ta_{n1} \\ x_2 & x_3 & x_4 & ta_{12} & ta_{22} & ... & ta_{n2} \\ x_3 & x_4 & x_5 & ta_{13} & ta_{23} & ... & ta_{nm} \end{Bmatrix}$$

With its target vector remaining the same:

$$\mathbf{y} = \begin{Bmatrix} y_1 \\ y_2 \\ y_3 \end{Bmatrix}$$

STILL PENDING...

The thought process behind our sliding window approach has to do with the data and the following: The dataset is NOT stationary. The data is not independent and identically distributed.

Non Stationary:

- Upwards trend in price: price is moving up with time according to inflation and such

- Unexpected Global events: war resulted in gas prices going way up

- Market volatility: Periods of market instability, increased renewable generation (which can lead to more volatile prices due to intermittency), or geopolitical events can cause price swings to be larger or smaller at different times.

Not IID Independent and identically distributed:

- Because of the non-stationarity (trends, seasonality, changing volatility), the probability distribution from which your price data points are drawn is not the same across all observations. For example, the distribution of prices in summer will be different from winter, and the distribution during a period of high gas prices will differ from a period of low gas prices.

This initially consisted of just the *MarginalES* values, as the sliding window width desired for that training batch:

```
import pandas as pd
import numpy as np
import os

def create_weight_matrix(data, window_size):
    """
    Creates a weight matrix where each row is a sliding window of prices,
    and a corresponding target vector containing the next price value.

    Parameters:
    ----------
    dataframe : pandas.DataFrame
        DataFrame only containing at 'Datetime' and 'MarginalES' columns,
    window_size : int
        Number of historical price points to include in each window

    Returns:
    -------
    X : pandas.DataFrame
```

```
        DataFrame with sliding windows as rows
    y : pandas.Series
        Series with target values (next price after each window)
    """

    # Extract the MarginalES column (assuming it's the one we want)
    if 'MarginalES' in data.columns:
        prices = data['MarginalES'].values
    else:
        # Assume it's the second column (index 1)
        prices = data.iloc[:, 1].values

    # Create empty matrices
    X = np.zeros((len(prices) - window_size - 1, window_size))
    y = np.zeros(len(prices) - window_size - 1)

    # Fill the matrices with the sliding windows and targets
    for i in range(len(prices) - window_size - 1):
        X[i, :] = prices[i:i+window_size]  # Window of prices
        y[i] = prices[i+window_size]       # Next price after window

    # Convert to DataFrame/Series for easier use in training
    return pd.DataFrame(X), pd.Series(y)
```

This was later modified to include extra feature columns:

```
import pandas as pd
import numpy as np
import os

def create_weight_matrix_with_features(dataframe, window_size, debug
    =False):
    """
    Creates a sliding window dataset for time series forecasting
        where each row contains:
    1. A window of historical prices (right-aligned)
    2. Feature values from the most recent point in the window
    3. Target value (next price after the window)

    Parameters:
    ----------
    dataframe : pandas.DataFrame
        DataFrame containing at minimum 'Datetime' and 'MarginalES'
            columns,
        plus any additional feature columns
    window_size : int
        Number of historical price points to include in each window
    debug : bool, optional
        If True, print debugging information instead of using logger

```

```
    Returns:
    -------
    X : pandas.DataFrame
        Features DataFrame with:
        - Historical prices labeled as 'price_t-n' through 'price_t-1'
        - All additional features from the original dataframe
    y : pandas.Series
        Target values (price at time t)
    """

    # Input validation
    if window_size < 1:
        raise ValueError("Window size must be at least 1")
    if len(dataframe) <= window_size:
        raise ValueError(f"DataFrame must have more rows ({len(dataframe)}) than window_size ({window_size})")
    if 'Datetime' not in dataframe.columns or 'MarginalES' not in dataframe.columns:
        raise ValueError("DataFrame must contain 'Datetime' and 'MarginalES' columns")

    X, y = [], []

    # Extract price data and features
    df_prices = dataframe[['Datetime', 'MarginalES']]
    df_features = dataframe.iloc[:, 2:]  # Exclude 'Datetime' and 'MarginalES'
    feature_names = df_features.columns.tolist()

    if debug:
        print(f"Feature columns identified: {feature_names}")

    # Create samples from the data
    for i in range(window_size, len(df_prices)):
        # Extract sliding window for prices (right-aligned)
        window = df_prices.iloc[i-window_size:i, 1:].values.flatten()

        # Extract corresponding feature row (from the most recent point in the window)
        feature_row = df_features.iloc[i-1].values.flatten()

        # Concatenate sliding window prices with feature row
        X.append(np.concatenate((window, feature_row)))
        y.append(df_prices.iloc[i, 1])  # Next price point as target

    # Create column names for the price window
    price_columns = [f'price_t-{window_size-i}' for i in range(window_size)]

```

```
    # Return DataFrame with proper column names
    X_df = pd.DataFrame(X, columns=price_columns + feature_names)
    y_series = pd.Series(y, name='price_t')

    if debug:
        print(f"X DataFrame shape: {X_df.shape}")
        print(f"Feature columns count: {len(feature_names)}")
        print(f"Price window columns: {price_columns}")
        print(f"Total columns: {len(X_df.columns)}")
        print(f"Sample size: {len(X_df)}")

    return X_df, y_series
```

For an example run we can retrieve the database and select a date range:

```
import pandas as pd
from utils.sliding_window import create_weight_matrix,
    create_weight_matrix_with_features

# read data
csv_hour_file = '../data/hour_14_metrics.csv'
df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])
df = df[['Datetime', 'MarginalES']]

# Date range for the training matrix
train_start_date = '2018-12-25'
train_end_date = '2022-01-01'

train_subset_df = df[(df['Datetime'] >= train_start_date) & (df['
    Datetime'] <= train_end_date)]

# Sliding window size
window_size = 3

# Create sliding window matrix - method from utils.sliding_window
X_train, y_train = create_weight_matrix(train_subset_df, window_size
    )
```

We may confirm the matrix is correctly built with the following results:

```
print(X_train.head())
       0      1      2
0  66.58  67.20  68.12
1  67.20  68.12  64.64
2  68.12  64.64  57.39
3  64.64  57.39  63.91
4  57.39  63.91  65.22

print(y_train.head())
```

```
0    64.64
1    57.39
2    63.91
3    65.22
4    65.88
```

3.4. Machine Learning Model Training

- Linear Regression: Linear combination of every feature

- Lasso Regression: Can turn unimportant features into 0s

- Random Forest: Feature engineering

Due to our particular system, the training will consist of complete rows of that feature matrix.

Cross validation?? Not really done, in another way w the sliding window cambio parametros pero no la arquitectura del modelo

Review the mini model theory, why is it reasonable to not use training set, validation and test sets?

Contar los bloques empleados en cada parte – como junto las piezas para construir el sistema – porque uso las piezas que uso

Hablar de implementaciones como que librerías, que scripts etc – no es muy necesario, mas importante la idea

4. EXPERIMENTATION

This chapter is dedicated to reviewing all of the relevant information pertaining the experimentation phase of the project. We will review the specifics of the data that was dealt with, and explain the decisions taken for the preparation of such data, and the training of the different machine learning models employed. We will review all of the results of the different models, and discuss in depth the different set-up variations for the models.

4.1. Data Description

The data that was used for this project was exlcusively publicly available information, published by the OMIE, which stands for *Operador del Mercado Ibérico de Energía*, meaning the Iberian operator of the energy market. The data that was retrieved for the project takes the following shape:

```
MARGINALPDBC;
2018;01;01;1;28.1;6.74;
2018;01;01;2;33;4.74;
2018;01;01;3;32.9;3.66;
2018;01;01;4;28.1;2.3;
2018;01;01;5;27.6;2.3;
2018;01;01;6;24.6;2.06;
2018;01;01;7;20.1;2.06;
2018;01;01;8;19.9;2.06;
2018;01;01;9;19.84;2.3;
2018;01;01;10;19.9;2.3;
2018;01;01;11;19.9;2.3;
2018;01;01;12;19.9;2.3;
2018;01;01;13;23.6;2.3;
2018;01;01;14;25.1;2.3;
2018;01;01;15;23.6;5;
2018;01;01;16;24.6;5;
2018;01;01;17;25.1;5;
2018;01;01;18;27.6;8.85;
2018;01;01;19;27.6;15.93;
2018;01;01;20;28.1;22.02;
2018;01;01;21;32.9;20;
2018;01;01;22;28.1;21.95;
2018;01;01;23;28.1;23.52;
2018;01;01;24;27.6;16.35;
*
```

We can understand and interpret this format following the OMIE's guide: *Modelo de Ficheros para la distribución pública de Información del mercado de electricidad 1.35* [7]. In page number 67, chapter 6.18 we may find the following information regarding the encoding format for the information:

```
6.18 Precios marginales del mercado diario (MARGINALPDBC)

Fichero con los precios marginales del Mercado Diario para cada una de
las horas.
Nombre del fichero: marginalpdbc_aaaammdd.v donde aaaammdd
corresponde a la fecha de sesión y v es la versión del fichero.

Descripción de los campos:
CAMPO DESCRIPCIÓN VALORES VÁLIDOS
Año Año I4 - 20XX
Mes Mes I2 - 1 a 12
Día Día I2 - 1 a 31
Hora Hora I2 - 1 a 25
MarginalPT Precio marginal zona Portuguesa F8.2 - -99999.99 a 99999.99
MarginalES Precio marginal zona Española F8.2 - -99999.99 a 99999.99
```

From this data description we can obtain the following column names: *Año, Mes, Día, Hora, MarginalPT* and *MarginalES*. As a proof of concept, in this investigation we will be focusing on a single time slot, simplifying the data, from multiple data points per day, to a single one. We are exclusively interested in the value of the last column, *MarginalES*, and specifically the row pertaining the 14$^{\text{th}}$ hour of each day, which we will refer to as 14H.

Data analysis: As previously commented: the in the No estacionario.

No son datos IID - Independientes e identicamente distribuidos.

Hay correlacion en mis datos entre cada dia? Si deberia ser relativamente alta, estacional, pero progresiva y lenta.

For the data visualization portion, I employed various methods, bot plotting values with matplotlib, seaborn and using the Microsoft Data Wrangler Visual Studio Code extension.

Explain some more about the data distribution.

Explain the ups and downs of the data (war and relation to natural gas prices).

Explain the zero price.

4.2. Model Set Up Explanation

How do I use the previously explained system - what parameters did I modify to obtain the final results. Feature engineering, Alpha variation testing in Lasso, Tree depth and number of leafs. Que modelos y que configs

- Linear Regressor
- Lasso Regressor
- Random Forest

4.3. Results of the Testing

Results as graph/ tables - This would be to compare a model across various iterations. Model best-run comparisons, etc.

Review error rates and compare with other iterations/ batches and other models.

Error medio en todas las predicciones

Y percentiles (expectation shortfall tambien?)

4.4. Low Level Discussion

This would be the final explanation that I would give to a colleague of mine, with full details on how I have done everything

5. REGULATORY FRAMEWORK

Not essential for this investigative project because we do not go to market.

5.1. Data Availability

Habria que hablar de las normas, pq si alguien lo usa es para ir a mercado

Current Spanish and European regulations allow for the use of publicly available institutional data for use in educational investigative work.

I need to talk about laws, what data is permissible to use and how. This would be essential for a project that does indeed go to market.

For us, its not that relevant.

5.2. Software & Licenses

For the development of this project I have used a variety of open and closed source utilies and code.

For the sake of organization, I will categorize this section into two distinct parts, required software to reproduce the project, and optional software for ease of production, plannification, organization or otherwise related.

Required:

Python - This was the programming language in wich the entirety of the project was developed [8]

Scikit-learn - library for ml [9]

Pandas - data management [10]

ta library - Technical Analysis library[11]

Light use of Seaborn - data visualization [12]

NumPy - number manipulation [13]

macOS / Windows - closed source platforms across where the project was developed [14] & [15]

Optional:

Git - a distributed version control system [16]

GitHub - Repository hosting platform [17]

Visual Studio Code - open source text editor for my general coding needs [18]

Visual Studio Code extensions such as Microsoft Data Wrangler, Python packages or such

Python Virtual Environment (venv) - This was created as it is best practice to create new Python virtual environments for each new project, as specially in a production environment these usually require certain specific versions that must be met in order for all components to work as intended. An alternative method of dealing with this would be using Conda, and creating with it a new Conda environment, [3]

Overleaf - closed source platform used to compile LaTeX code [19]

LaTeX - open source language used to create the final project document [20]

OpenAI ChatGPT, Google Gemini & Anthropic Claude - closed source large language models used to aid in troubleshooting general coding issues

6. SOCIO-ECONOMIC ENVIRONMENT

6.1. Socio-economic Impact

Why is this relevant?

- Energy Prediction

- Shutdowns

- Price sensitivity

6.2. Project plan

TODO: Gantt diagram representing time spent in each phase - Design and innovation time frames

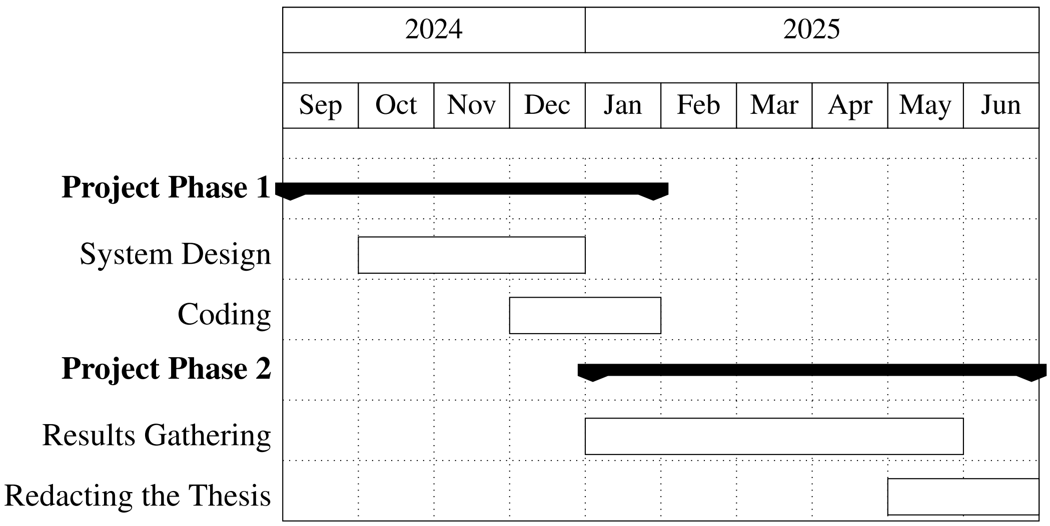

Fig. 6.1. Gantt Diagram of the Project Lifecycle

6.3. Budget

This section will consist of a complete price breakdown of the investigation, consisting of material costs such as the equipment utilized, and the human capital that composed the investigation team. It is notable to highlight that as previously mentioned, no paid license of any sort was obtained for the realization of this project. All of the programs, tools or code, were either *Open Source* or free to use software under exclusive *non-commercial use* licenses.

The most basic requirement for the physical needs of this project is a computer with an desktop operating system. Since the project was written in Python [8], it would be reasonable to consider it platform agnostic. Any computer capable of running any recent operating system, configured with a recent version of Python would be able to run this project. This includes any of the three most popular desktop operating systems, Windows, macOS and any Linux distribution.

The project was mainly tested using Python 3.13 [2] which does require a recent operating system such as macOS 13 or Windows 10. But as for the project itself, there is no platform, processor architecture, or processing power requirements. In my case, I mainly developed the code in my personal laptop, a 2021 MacBook Pro configured with the ARM M1 Pro chip and 16GB of RAM, running the latest available macOS version at this time, macOS 15 [14]. This laptop was obtained from a second-hand store circa June 2024, mainly for personal use, but also helping keep the theoretical project costs down.

The following table outlines the specific equipment costs incurred for the development of this investigative project:

Equipment	**Price (€)**	**Amortization per year (€)**	**Useful Life (years)**	**Usage Time (months)**	**Cost (€)**
MacBook Pro M1 Pro	1250	312.5	4	9.5	247.4

TABLE 6.1. EQUIPMENT AMORTIZATION

Regarding the human capital that was required for the successful development of this project, it will consist exclusively on two people. The director of my bachelor's thesis, professor Emilio Parrado Hernández, PhD, as the expert Machine Learning consultant, and myself as a junior software engineer.

The following table displays the estimated hourly rates of each person, the hours dedicated to the project, and the total cost:

Name	**Role**	**Price (€/hour)**	**Hours**	**Cost (€)**
Emilio Parrado Hernández	Expert ML Consultant (PhD)	200	-	-
Rodrigo De Lama Fernández	Junior Engineer (Pre-Graduate)	30	-	-

TABLE 6.2. HUMAN COSTS

For our final budgeting considerations, other miscellaneous expenses, such as transportation costs to the university for on site meetings with my bachelor's thesis director. Summing up all costs, the following table estimates what this project would have cost to investigate:

Concept	**Cost (€)**
Direct/Material Costs	247.4
Engineering Costs	-
University Carlos III Costs - Expert ML Consultant	-
Total	**-**

TABLE 6.3. FINAL COST BREAKDOWN

7. CONCLUSIONS

In this investigation we have analyzed in depth an innovative methodology for predictive modeling of energy prices with Machine Learning algorithms, enhanced by the usage of technical analysis indicators in feature engineering, in order to enhance prediction precision. This final chapter will review the general conclusions attained throughout the development of this project's Machine Learning based solution for energy price predictions. It will also discuss potential development paths for future work pertaining the system designed in this investigative project

7.1. Review of the Investigation

After finishing the whole project, read it, and reintroduce the objectives to the reader. Remind them of the completion of them

7.2. Revisiting the Objectives

Predicting the prices in an accurate manner using Machine Learning algorithms and technical analysis

7.3. Future work

- Less absolute error

- Better overall precision: better percentile accuracy

BIBLIOGRAPHY

[1] EDEM, *Cómo se fija el precio de la electricidad en españa*, https://edem.eu/como-se-fija-el-precio-de-la-electricidad-en-espana-para-dummies/, Accessed on June 5, 2025, Jun. 2025.

[2] Python Software Foundation, *Python 3.13*, https://www.python.org/downloads/release/python-3130/, 2025.

[3] Python Software Foundation, *Venv — creation of virtual environments*, https://docs.python.org/3/library/venv.html, 2025.

[4] Anaconda Inc., *Miniconda*, https://www.anaconda.com/docs/getting-started/miniconda/main, 2025.

[5] OMIE, *Precios horarios del mercado diario en españa*, https://www.omie.es/es/file-access-list?parents=/Mercado%20Diario/1.%20Precios&dir=Precios%20horarios%20del%20mercado%20diario%20en%20EspaÃśa&realdir=marginalpdbc, Accessed on June 5, 2025, Jun. 2025.

[6] The Pandas Development Team, *Pandas dataframe*, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html, 2025.

[7] OMIE, *Modelo de ficheros para la distribución pública de información del mercado de electricidad 1.35*, https://www.omie.es/sites/default/files/2024-06/Formato_ficheros_inf_pub_135_0.pdf, Accessed on June 5, 2025, Jun. 2024.

[8] Python Software Foundation, *Python programming language*, https://www.python.org/about/, 2025.

[9] The scikit-learn Development Team, *Scikit-learn: Machine learning in python*, https://scikit-learn.org/stable/, 2025.

[10] The Pandas Development Team, *Pandas: Python data analysis library*, https://pandas.pydata.org, 2025.

[11] bukosabino, *Technical analysis library in python (ta)*, https://github.com/bukosabino/ta, 2025.

[12] Michael Waskom et al., *Seaborn: Statistical data visualization*, https://seaborn.pydata.org, Used lightly for visualization, 2025.

[13] The NumPy Development Team, *Numpy: Array programming for scientific computing*, https://numpy.org, 2025.

[14] Apple Inc., *Macos sequoia*, https://www.apple.com/macos/macos-sequoia/, 2025.

[15] Microsoft Corpotation, *Windows 11*, https://www.microsoft.com/windows/windows-11, 2025.

[16] Git, *Git: A distributed version control system*, https://git-scm.com, 2025.

[17] GitHub, *Github: Repository hosting platform*, https://github.com/about, 2025.

[18] Microsoft Corporation, *Visual studio code*, https://code.visualstudio.com/docs/editor/whyvscode, Open source code editor, 2025.

[19] Overleaf, *Overleaf: Online LaTeX editor*, https://www.overleaf.com/about, 2025.

[20] LaTeX Project, *Latex: A document preparation system*, https://www.latex-project.org, 2025.